

Leading GBM Model Testing File

Train SIPP model, score ACS, validate target-like thresholds, and export Stata predictions

```
# =====  
# PURPOSE  
# =====  
# Single-file testing workflow:  
#   1. Train the leading GBM model in SIPP.  
#   2. Define quartiles using the target-like SIPP test sample.  
#   3. Score the ACS target sample.  
#   4. Export ACS_SIPP_gbm.csv for Stata.  
#  
# The donor-sample restriction is written as an explicit filter below so it is  
# easy to test alternatives without tracing helper functions.  
# =====  
  
report_timestamp <- format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z")  
cat("Report rendered at:", report_timestamp, "\n")
```

Report rendered at: 2026-05-17 08:53:04 PDT

```
# Packages  
library(dplyr)  
library(readr)  
library(gbm)  
library(caret)  
library(pROC)  
library(knitr)
```

```
# =====  
# PATHS AND FILE NAMES  
# =====
```

```

if (Sys.info()["sysname"] == "Windows") {
  base_dir <- "G:/Shared drives/Undocu Research"
} else {
  base_dir <- "~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research"
}

data_path <- file.path(base_dir, "Data")
ml_output_path <- file.path(data_path, "ML Outputs")
dir.create(ml_output_path, showWarnings = FALSE, recursive = TRUE)

sipp_file <- file.path(data_path, "(Step 1 output) Core_TM SIPP 2008 Wave 2.csv")
acs_file <- file.path(data_path, "ACS Target Sample.csv")

# Stata-facing export
acs_prediction_export <- file.path(data_path, "ACS_SIPP_gbm.csv")

# Optional reproducibility objects
leading_gbm_model_file <- file.path(ml_output_path, "leading_gbm_model.rds")
leading_gbm_metadata_file <- file.path(ml_output_path, "leading_gbm_metadata.rds")

# =====
# MODEL SETTINGS
# =====

seed <- 1

gbm_n_trees <- 3500
gbm_shrinkage <- 0.005
gbm_interaction_depth <- 6
gbm_cv_folds <- 10

# =====
# MODEL FEATURES AND FORMULA
# =====
# Because the donor sample below is restricted to employed respondents, employed
# is intentionally not included as a predictor.
# =====

features <- c(
  "age",
  "fem",

```

```

"married",
"cit_spouse",
"medicaid",
"nonfluent",
"spanish_hispanic_latino",
"central_latino",
"bpl_asia",
"household_size",
"poverty",
"asian",
"black",
"white",
"other_race",
"years_us",
"years_us_missing",
"yrsted"
)

gbm_formula <- undocu_likely ~
  age +
  fem +
  married +
  cit_spouse +
  medicaid +
  nonfluent +
  spanish_hispanic_latino +
  central_latino +
  bpl_asia +
  household_size +
  poverty +
  asian +
  black +
  white +
  other_race +
  years_us +
  years_us_missing +
  yrsted

```

```

# =====
# READ CLEANED SIPP AND ACS FILES
# =====

```

```
sipp_raw <- read_csv(sipp_file, show_col_types = FALSE)
acs_raw <- read_csv(acs_file, show_col_types = FALSE)

tibble(
  file = c("SIPP donor file", "ACS target file"),
  raw_rows = c(nrow(sipp_raw), nrow(acs_raw))
) %>%
  kable()
```

file	raw_rows
SIPP donor file	5753
ACS target file	185845

```
# =====
# PREPARE MODEL SAMPLES
# =====
# EDIT THE SIPP FILTER HERE to test alternative donor samples.
# Current preferred donor sample: employed SIPP respondents ages 18-55.
# The ACS target sample is already employed, college-educated, and age-restricted
# in the Stata cleaning pipeline, so no extra substantive ACS filter is added here.
# =====

sipp <- sipp_raw %>%
  mutate(
    # SIPP outcome may be stored as "X1", "1", or numeric 1 depending on export.
    undocu_likely = ifelse(undocu_likely %in% c("X1", "1", 1), 1, 0)
  ) %>%
  filter(
    age >= 18,
    age <= 55,
    employed == 1
    # To test college-only donor training, add: , college == 1
    # To test age 22-55 donor training, change age >= 18 to age >= 22
  )

# Drop rows that are missing the outcome, any model feature, or college.
# college is retained because it defines the target-like SIPP validation sample.
sipp <- sipp[
  complete.cases(sipp[, c("undocu_likely", features, "college"))],
]
```

```

acs <- acs_raw[
  complete.cases(acs_raw[, c("year", "serial", "pernum", features)]),
]

tibble(
  sample = c("SIPP", "ACS"),
  raw_rows = c(nrow(sipp_raw), nrow(acs_raw)),
  model_rows = c(nrow(sipp), nrow(acs)),
  rows_dropped = c(nrow(sipp_raw) - nrow(sipp), nrow(acs_raw) - nrow(acs))
) %>%
  kable()

```

sample	raw_rows	model_rows	rows_dropped
SIPP	5753	3098	2655
ACS	185845	185845	0

```

# =====
# SPLIT SIPP INTO TRAINING AND TEST SAMPLES
# =====

set.seed(seed)

train_index <- createDataPartition(
  sipp$undocu_likely,
  p = 0.70,
  list = FALSE
)

train_gbm <- sipp[train_index, ]
test_gbm <- sipp[-train_index, ]

tibble(
  sample = c("Training SIPP", "Held-out SIPP test"),
  n = c(nrow(train_gbm), nrow(test_gbm)),
  positives = c(
    sum(train_gbm$undocu_likely == 1),
    sum(test_gbm$undocu_likely == 1)
  ),
  positive_share = c(
    mean(train_gbm$undocu_likely == 1),

```

```

    mean(test_gbm$undocu_likely == 1)
  )
) %>%
  kable(digits = 4)

```

sample	n	positives	positive_share
Training SIPP	2169	682	0.3144
Held-out SIPP test	929	311	0.3348

```

# =====
# TRAIN THE LEADING GBM MODEL
# =====

set.seed(seed)

gbm_model <- gbm(
  formula = gbm_formula,
  data = train_gbm,
  distribution = "bernoulli",
  verbose = FALSE,
  cv.folds = gbm_cv_folds,
  n.trees = gbm_n_trees,
  shrinkage = gbm_shrinkage,
  interaction.depth = gbm_interaction_depth
)

optimal_trees_gbm <- gbm.perf(
  gbm_model,
  method = "cv",
  plot.it = FALSE
)

tibble(
  setting = c("Maximum trees", "Optimal trees selected by CV", "Shrinkage", "Interaction depth", "CV folds"),
  value = c(gbm_n_trees, optimal_trees_gbm, gbm_shrinkage, gbm_interaction_depth, gbm_cv_folds)
) %>%
  kable(digits = 4)

```

setting	value
Maximum trees	3500.000
Optimal trees selected by CV	786.000
Shrinkage	0.005
Interaction depth	6.000
CV folds	10.000

```
# Variable importance: descriptive, not causal.
var_importance <- summary(gbm_model, plotit = FALSE)
kable(var_importance, digits = 3)
```

	var	rel.inf
age	age	22.504
years_us	years_us	18.720
yrsed	yrsed	13.704
household_size	household_size	11.596
central_latino	central_latino	9.358
cit_spouse	cit_spouse	4.094
nonfluent	nonfluent	2.683
poverty	poverty	2.648
fem	fem	2.647
spanish_hispanic_latino	spanish_hispanic_latino	2.060
married	married	2.013
white	white	1.760
black	black	1.171
asian	asian	1.164
medicaid	medicaid	1.150
bpl_asia	bpl_asia	1.035
years_us_missing	years_us_missing	0.951
other_race	other_race	0.743

```
# =====
# PREDICT PROBABILITIES IN SIPP TEST AND ACS TARGET SAMPLES
# =====

test_gbm$gbm_undocu_p <- predict(
  gbm_model,
  newdata = test_gbm,
  n.trees = optimal_trees_gbm,
```

```

    type = "response"
  )

  acs$gbm_undocu_p <- predict(
    gbm_model,
    newdata = acs,
    n.trees = optimal_trees_gbm,
    type = "response"
  )

  bind_rows(
    test_gbm %>%
      summarise(
        sample = "Held-out SIPP test",
        n = n(),
        mean_probability = mean(gbm_undocu_p),
        p25 = quantile(gbm_undocu_p, .25),
        median = median(gbm_undocu_p),
        p75 = quantile(gbm_undocu_p, .75)
      ),
    acs %>%
      summarise(
        sample = "ACS target",
        n = n(),
        mean_probability = mean(gbm_undocu_p),
        p25 = quantile(gbm_undocu_p, .25),
        median = median(gbm_undocu_p),
        p75 = quantile(gbm_undocu_p, .75)
      )
  ) %>%
    kable(digits = 4)

```

sample	n	mean_probability	p25	median	p75
Held-out SIPP test	929	0.3076	0.1703	0.2553	0.4571
ACS target	185845	0.2242	0.1439	0.1946	0.2755

```

# =====
# DEFINE OFFICIAL QUANTILES USING TARGET-LIKE SIPP TEST SAMPLE
# =====
# Threshold reference sample: SIPP test observations that match the ACS target

```

```

# population: college graduates, employed, ages 22-55.
# =====

sipp_targetlike_test <- test_gbm %>%
  filter(
    college == 1,
    employed == 1,
    age >= 22,
    age <= 55
  )

targetlike_breaks <- quantile(
  sipp_targetlike_test$gbm_undocu_p,
  probs = c(0, .25, .50, .75, 1),
  na.rm = TRUE
)

targetlike_breaks[1] <- -Inf
targetlike_breaks[5] <- Inf

sipp_targetlike_test$gbm_undocu_q <- cut(
  sipp_targetlike_test$gbm_undocu_p,
  breaks = targetlike_breaks,
  include.lowest = TRUE,
  labels = c("Q1", "Q2", "Q3", "Q4")
)

acs$gbm_undocu_q <- cut(
  acs$gbm_undocu_p,
  breaks = targetlike_breaks,
  include.lowest = TRUE,
  labels = c("Q1", "Q2", "Q3", "Q4")
)

tibble(
  cutoff = c("Q1/Q2", "Q2/Q3", "Q3/Q4"),
  probability_cutoff = c(targetlike_breaks[2], targetlike_breaks[3], targetlike_breaks[4])
) %>%
  kable(digits = 4)

```

cutoff	probability_cutoff
Q1/Q2	0.1464
Q2/Q3	0.2057
Q3/Q4	0.3078

```
targetlike_sipp_quartiles <- sipp_targetlike_test %>%
  group_by(gbm_undocu_q) %>%
  summarise(
    n = n(),
    positives = sum(undocu_likely == 1),
    precision = mean(undocu_likely == 1),
    mean_probability = mean(gbm_undocu_p),
    .groups = "drop"
  )

acs_quartiles <- acs %>%
  group_by(gbm_undocu_q) %>%
  summarise(
    n = n(),
    share = n() / nrow(acs),
    mean_probability = mean(gbm_undocu_p),
    .groups = "drop"
  )

kable(targetlike_sipp_quartiles, digits = 4)
```

gbm_undocu_q	n	positives	precision	mean_probability
Q1	51	2	0.0392	0.1163
Q2	51	9	0.1765	0.1737
Q3	51	12	0.2353	0.2423
Q4	51	30	0.5882	0.4300

```
kable(acs_quartiles, digits = 4)
```

gbm_undocu_q	n	share	mean_probability
Q1	48947	0.2634	0.1198
Q2	52240	0.2811	0.1744
Q3	49001	0.2637	0.2482
Q4	35657	0.1919	0.4074

gbm_undocu_q	n	share	mean_probability
--------------	---	-------	------------------

```
# =====
# DEFINE HIGH-RECALL THRESHOLD USING TARGET-LIKE SIPP TEST SAMPLE
# =====
# The threshold is the lowest predicted probability among the top 75% of true
# positives in the target-like SIPP test sample.
# =====

top_75_targetlike <- sipp_targetlike_test %>%
  filter(undocu_likely == 1) %>%
  arrange(desc(gbm_undocu_p)) %>%
  mutate(
    rank = row_number(),
    cumulative_share_of_positives = rank / n(),
    top_75 = cumulative_share_of_positives <= 0.75
  ) %>%
  filter(top_75 == TRUE)

threshold_75_gbm <- min(top_75_targetlike$gbm_undocu_p)

tibble(
  threshold = "Target-like high recall",
  probability_cutoff = threshold_75_gbm
) %>%
  kable(digits = 4)
```

threshold	probability_cutoff
Target-like high recall	0.218

```
# =====
# MODEL VALIDATION: AUC AND LIFT
# =====

auc_full <- pROC::auc(
  pROC::roc(
    response = test_gbm$undocu_likely,
    predictor = test_gbm$gbm_undocu_p,
    quiet = TRUE
  )
)
```

```

    )
  )

auc_targetlike <- pROC::auc(
  pROC::roc(
    response = sipp_targetlike_test$undocu_likely,
    predictor = sipp_targetlike_test$gbm_undocu_p,
    quiet = TRUE
  )
)

tibble(
  validation_sample = c("Full SIPP test", "Target-like SIPP test"),
  auc = c(as.numeric(auc_full), as.numeric(auc_targetlike))
) %>%
  kable(digits = 4)

```

validation_sample	auc
Full SIPP test	0.7567
Target-like SIPP test	0.7951

```

targetlike_base_rate <- mean(sipp_targetlike_test$undocu_likely == 1)

targetlike_lift <- targetlike_sipp_quartiles %>%
  mutate(
    base_rate = targetlike_base_rate,
    lift = precision / base_rate
  )

kable(targetlike_lift, digits = 4)

```

gbm_undocu_q	n	positives	precision	mean_probability	base_rate	lift
Q1	51	2	0.0392	0.1163	0.2598	0.1509
Q2	51	9	0.1765	0.1737	0.2598	0.6792
Q3	51	12	0.2353	0.2423	0.2598	0.9057
Q4	51	30	0.5882	0.4300	0.2598	2.2642

```

# =====
# PROBABILITY DISTRIBUTION CHECK
# =====

prob_distribution_check <- bind_rows(
  test_gbm %>%
    mutate(sample = "Full SIPP test") %>%
    select(sample, gbm_undocu_p),
  sipp_targetlike_test %>%
    mutate(sample = "Target-like SIPP test") %>%
    select(sample, gbm_undocu_p),
  acs %>%
    mutate(sample = "ACS target") %>%
    select(sample, gbm_undocu_p)
) %>%
  group_by(sample) %>%
  summarise(
    n = n(),
    mean_probability = mean(gbm_undocu_p),
    p10 = quantile(gbm_undocu_p, .10),
    p25 = quantile(gbm_undocu_p, .25),
    p50 = quantile(gbm_undocu_p, .50),
    p75 = quantile(gbm_undocu_p, .75),
    p90 = quantile(gbm_undocu_p, .90),
    .groups = "drop"
  )

kable(prob_distribution_check, digits = 4)

```

sample	n	mean_probability	p10	p25	p50	p75	p90
ACS target	185845	0.2242	0.1147	0.1439	0.1946	0.2755	0.3892
Full SIPP test	929	0.3076	0.1220	0.1703	0.2553	0.4571	0.5371
Target-like SIPP test	204	0.2406	0.1100	0.1464	0.2057	0.3078	0.4568

```

# =====
# EXPORT ACS PREDICTIONS FOR STATA
# =====

acs_gbm_export <- acs %>%

```

```

mutate(
  gbm_high_prob = ifelse(gbm_undocu_q == "Q4", 1, 0),
  gbm_low_prob = ifelse(gbm_undocu_q == "Q1", 1, 0),
  gbm_high_recall = ifelse(gbm_undocu_p >= threshold_75_gbm, 1, 0)
) %>%
select(
  gbm_undocu_p,
  gbm_undocu_q,
  gbm_high_prob,
  gbm_low_prob,
  gbm_high_recall,
  year,
  serial,
  pernum
)

write_csv(acs_gbm_export, acs_prediction_export)

tibble(
  output = "Official ACS prediction export for Stata",
  path = acs_prediction_export,
  rows = nrow(acs_gbm_export)
) %>%
kable()

```

output	path	rows
Official ACS prediction export for Stata	~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research/Data/ACS_SIPP_gbm.csv	185845

```

# =====
# SAVE MODEL AND METADATA
# =====

model_metadata <- list(
  model_name = "leading_gbm",
  threshold_definition = "target-like SIPP test sample: college graduates, employed, ages 22-29",
  donor_sample = "SIPP age 18-55, employed == 1",
  report_timestamp = report_timestamp,
  features = features,

```

```

seed = seed,
gbm_n_trees = gbm_n_trees,
gbm_shrinkage = gbm_shrinkage,
gbm_interaction_depth = gbm_interaction_depth,
gbm_cv_folds = gbm_cv_folds,
optimal_trees_gbm = optimal_trees_gbm,
targetlike_breaks = targetlike_breaks,
threshold_75_gbm = threshold_75_gbm,
train_n = nrow(train_gbm),
test_n = nrow(test_gbm),
targetlike_test_n = nrow(sipp_targetlike_test),
full_sipp_auc = as.numeric(auc_full),
targetlike_sipp_auc = as.numeric(auc_targetlike),
created_at = Sys.time()
)

saveRDS(gbm_model, leading_gbm_model_file)
saveRDS(model_metadata, leading_gbm_metadata_file)

tibble(
  object = c("Fitted leading GBM model", "Leading GBM metadata"),
  path = c(leading_gbm_model_file, leading_gbm_metadata_file)
) %>%
  kable()

```

object	path
Fitted leading GBM model	~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research/Data/ML Outputs/leading_gbm_model.rds
Leading GBM metadata	~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research/Data/ML Outputs/leading_gbm_metadata.rds